

AgentDescent: Gradient Descent Where the Parameters Are Agents

A Parallel, Asynchronous, Merge-Based Framework for Self-Evolving Agent Artifacts

Birfy Chen

birfychen@gmail.com

<https://github.com/Birfy/agentdescent>

August 2026

Abstract

Self-improving agent systems evolve their own skills, prompts, and harnesses, but almost universally through a *serial* loop — one candidate proposed, evaluated, and accepted or rejected per iteration — bounding improvement throughput at $O(1/T_{\text{iter}})$. We present **AgentDescent**, a framework that transplants the distributed-training stack onto agent self-evolution: artifacts play the role of parameters, diffs with attached evidence play the role of gradients, a version-vectored ledger plays the role of the parameter server, and the optimizer step is a merge decision. Because diffs, unlike gradients, do not add, aggregation is performed as a discrete-space optimization: conflict resolution, fusion, and statistical acceptance under a per-artifact Beta posterior. We analyze efficiency as three composable sources of speedup: (i) overlap of I/O-bound rollouts, (ii) barrier-free asynchronous execution under per-diff bounded staleness, and (iii) *reduced validation cost* — held-out evaluation dominates the critical path, and merging N concurrent proposals requires a single acceptance test, a saving that *reflective merge* extends to artifacts whose proposals conflict. Eighteen published self-evolution algorithms are ported to the engine as plug-ins rather than forks and run unchanged under serial, synchronous, and asynchronous scheduling. On live models, removing the round barrier raises effective concurrency from $1.85\times$ to $3.22\times$ over a faithful serial control at a pinned budget; merged validation reduces model calls by 28%; and all ported algorithms retain their learning behavior under the most aggressive schedule. An equal-budget comparison of merge-of- N against best-of- N selection does not yet separate the two, and we analyze why. Code and reproduction commands for every reported number are open source.

1 Introduction

A growing body of work shows that large-language-model agents can improve their own artifacts: prompts [1, 3], in-context playbooks [28], skill libraries [20, 30], agentic workflows [5, 27], and their own code [13, 14, 24, 26]. These systems largely share an execution structure inherited from the Gödel machine [16]: a serial improvement loop in which one candidate modification is proposed, evaluated, and accepted or rejected before the next is considered. If one iteration takes T_{iter} wall-clock seconds, improvement throughput is bounded at one accepted change per T_{iter} , independent of available compute.

Distributed deep learning encountered the same bottleneck and resolved it with a now-standard stack: data-parallel workers computing gradients concurrently, parameter servers aggregating them [9], staleness-tolerant asynchronous updates [12], per-parameter adaptive steps [8], and gradient surgery for conflicting updates [25]. This paper develops the thesis that the same stack, suitably reinterpreted, applies to agent self-evolution — and identifies precisely where the reinterpretation must depart from the original.

AgentDescent (Fig. 1) instantiates the mapping. A library of evolvable artifacts plays the role of the parameter tensor; a diff annotated with an *evidence card* (the rollout that motivated it) plays the role of a gradient; a git-backed, version-vectored ledger plays the role of the parameter server; and the optimizer step is an *aggregator* that merges concurrent diffs. N workers roll out tasks against snapshots of the library and propose diffs in parallel; an asynchronous merger aggregates them, targeting $O(N/T_{\text{iter}})$ improvement throughput.

The mapping is productive precisely because of where it breaks. *Gradients add; diffs do not*. Two gradients can always be summed; two textual edits to the same artifact may be complementary, redundant, or contradictory. Aggregation is therefore not averaging but a decision procedure — staleness filtering, conflict resolution, fusion, and a statistical acceptance test (Section 4). A second disanalogy is equally consequential: training code is not modifiable by the gradients it computes, whereas a self-evolving system can in principle rewrite its own verifier. AgentDescent therefore enforces a layered governance model in which safety-critical components are frozen (Section 6).

Contributions.

1. **A system design and a discrete-space aggregator** mapping the distributed-training stack onto parallel self-evolution — per-diff bounded staleness, conflict projection, fusion, Beta-posterior acceptance, dual-branch promotion — with a formal account of where the analogy holds and where it must break (Sections 3 and 4).
2. **An efficiency analysis identifying three composable sources of speedup** (Section 5): rollout overlap, barrier removal, and — specific to merge-based evolution — reduced validation cost, in which merging collapses N acceptance tests into one and *reflective merge* extends the reduction to artifacts whose concurrent proposals conflict.
3. **An extensibility mechanism** under which published algorithms are ported as plug-ins rather than forks (Sections 3.3 and 7): a diff-encoding strategy plus an optional `aggregator_factory`, or a declarative policy bundle. Eighteen ports run under all three schedulers without modification.
4. **A live-model evaluation** (Section 8): $1.85\times \rightarrow 3.22\times$ effective concurrency from barrier removal against a faithful serial control at a pinned budget; 28% fewer model calls from merged validation; an eleven-algorithm scheduler study; evidence that quality is preserved under the most aggressive schedule; and an equal-budget merge-versus-selection comparison reported, with analysis, as not yet statistically separated.

2 Related Work

Self-evolving agents span prompt evolution [1, 3], context and playbook evolution [28], skill libraries [20, 30], workflow and harness search [5, 27], self-modifying code [14, 24, 26], evolutionary program search [11, 13, 17], and label-free self-play [6, 22, 29], with reflection [10, 18] as a shared inner proposal mechanism. Nearly all publish a serial outer loop; AgentDescent treats that loop as one scheduling regime among three. On the systems side, AgentDescent borrows its vocabulary from distributed training: parameter servers [9], lock-free and bounded-staleness asynchrony [4, 12, 15], per-parameter adaptivity [8], gradient surgery [25], weight averaging [21], and tensor/pipeline parallelism [7, 19]. Concurrent work on parallel self-evolution (barrier-free evolution loops; co-evolutionary verification [2]) indicates that the throughput premise is an open hypothesis; our contribution is the specific synthesis — concurrent, staleness-bounded, conflict-resolved diff-level merge over a versioned ledger — together with the measurement harness required to evaluate it.

Table 1: The correspondence between distributed training and AgentDescent. The final row is a deliberate disanalogy, enforced rather than elided.

Model training	AgentDescent
parameter tensor θ	library of Evolvable artifacts
gradient g	Diff + EvidenceCard
parameter server	git-backed, version-vectored Ledger
optimizer step	Aggregator merge decision
per-parameter adaptive step (Adam)	per-artifact Beta-posterior acceptance
bounded staleness (async SGD/RL)	per-diff lag η + rebase re-verification
EMA / weight averaging	dev/stable dual branch
training code (not self-modifiable)	L0 frozen governance layer

3 Problem Setup and System Design

3.1 Problem formulation

Let $\mathcal{A}$ denote the space of *artifact libraries*: finite keyed collections $A = \{a_k\}$ of versioned textual or executable artifacts (skills, prompts, playbooks, harness modules, file trees). An agent parameterized by A induces a policy π_A ; given a task distribution $\mathcal{D}$ and a bounded reward $r \in [0, 1]$, the objective is

$$A^* = \arg \max_{A \in \mathcal{A}} \mathbb{E}_{x \sim \mathcal{D}} [r(\pi_A(x), x)], \quad (1)$$

optimized from rollout evidence only: $\mathcal{A}$ is discrete, and no gradient of r with respect to A exists. The serial baseline — propose one candidate A' , evaluate on a held-out split D_{val} , accept if better — performs one accepted modification per iteration time T_{iter} . AgentDescent’s premise is that with N concurrent proposers and merge-based aggregation, accepted-modification throughput can approach $O(N/T_{\text{iter}})$ without sacrificing the acceptance discipline.

A *diff* d is a keyed partial edit over A , produced from a rollout by a *strategy* S that maps a natural-language proposal to operations on keys (rule slots, playbook bullets, file paths). The key structure is what makes two diffs comparable: edits on disjoint keys *fuse*; edits on the same key *conflict*. Every diff carries an *evidence card* $e = (x, \tau, r, v_{\text{base}})$ recording the task, rollout, reward, and the library version the proposer observed.

3.2 System components

Table 1 summarizes the correspondence and Fig. 1 the resulting architecture. The **ledger** is the parameter server: a git-backed store with per-artifact version vectors, compare-and-swap (CAS) commits, and a dual dev/stable branch pair; *stable* is promoted after K regression-free rounds on dev, an EMA-style confirmation in which a new commit restarts the counter. **Workers** hold snapshots of the library, roll out tasks, and emit (diff, evidence) pairs into a thread-safe buffer. The **aggregator** consumes the buffer and performs merge decisions (Section 4). Version vectors make the staleness of any diff computable at merge time:

$$\eta(d) = \max_{a \in \text{touched}(d)} (v_{\text{head}}(a) - v_{\text{base}}(a)). \quad (2)$$

3.3 Pluggable seams

Each component in Fig. 1 sits behind an explicit interface, selected by one keyword argument of the single entry point `evolve()`: the *agent layer* (any prompt $\rightarrow$ text callable is a completion — Claude, OpenAI-compatible endpoints, a tool-using CLI agent, or plain `run/reward/propose` functions), the *strategy* (diff en-

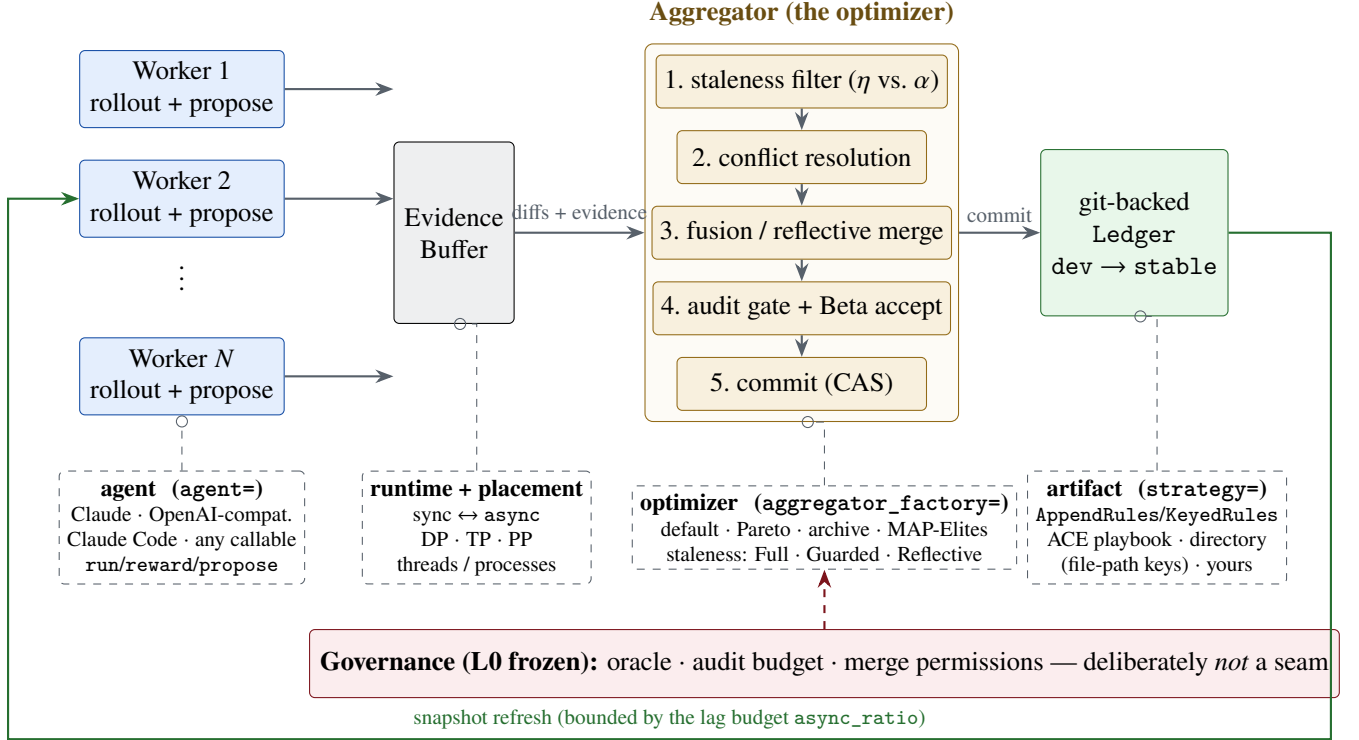


Figure 1: AgentDescent architecture. **Solid, top:** the fixed data path — N workers roll out tasks against ledger snapshots and emit diffs with evidence cards into a thread-safe buffer; a dedicated aggregator thread runs the five-stage merge pipeline and commits winners to the git-backed ledger; the green return edge is the only feedback path, bounded by the lag budget. **Dashed, bottom:** every component on the path is a *pluggable seam*, swapped by one keyword argument of `evolve()` — the model behind a worker, the whole optimizer, the staleness policy, the runtime and placement, and the artifact’s diff encoding. The eighteen ports of Section 7 are exactly such plug-ins. The one deliberate exception is the red governance layer: L0 (oracle, audit budget, merge permissions) is frozen and read-only to the loop, because a self-referential system must not be able to swap out its own judge.

coding), the *aggregator* (`aggregator_factory`), the *staleness policy*, the *runtime* (synchronous or barrier-free), the *parallelism scheme* (data/tensor/pipeline [7, 19]), and the *executor and sandbox*. Selection and acceptance rules are likewise named policy classes. Section 7 uses these seams to port eighteen published algorithms without modifying the engine. The single deliberate exception is governance (Section 6): the L0 layer exposes no seam, since a self-modifying system must not be able to replace its own evaluator.

4 The Aggregator as a Discrete-Space Optimizer

Evidence cards are bucketed per artifact; when a bucket reaches batch size B (or a timeout, so cold artifacts are not starved), the aggregator executes one optimizer step, summarized in Algorithm 1: **(1) staleness filtering** by Eq. (2) under the active policy (Section 5.2); **(2) conflict resolution** — syntactic (overlapping hunks) and semantic (contradictory operations) conflicts are detected, and contradictions are projected out in the manner of PCGrad [25], retaining the better member of each pair on a shared held-out subset; **(3) fusion** — complementary survivors are merged into one union candidate (cf. model soups [21]), with *reflective merge* handling contradictions that a keyed union cannot (Section 5.3); **(4) auditing** — merges with high blast radius or low trust are routed through an oracle with veto power; **(5) statistical acceptance and commit**. Let s and f be held-out successes and failures of the candidate relative to the incumbent. The candidate commits (by

CAS on dev) iff

$$\Pr[\Delta_a > 0 \mid s, f] > 1 - \delta_v, \quad \Delta_a \sim \text{Beta}(\alpha_0 + s, \beta_0 + f) - \frac{1}{2}, \quad (3)$$

a per-artifact posterior test rather than a point threshold — the counterpart of per-parameter adaptivity in Adam [8] — with δ_v annealed over artifact versions (the learning-rate-decay analogue) and a trust region capping diff size.

A natural objection to merge-based evolution is that two individually beneficial edits may be jointly harmful. Two properties bound the risk: the acceptance test Eq. (3) scores the *fused* candidate on the full held-out split, so a harmful fusion cannot commit; and the union is a superset of every constituent diff, so committing it discards no proposal. An optional *fusion tournament* additionally ranks the union against its constituents; it is disabled by default (it costs one additional held-out sweep per candidate against a recoverable gain) but serves as the measurement instrument for the objection, reporting win rate, the losing tail, and a *contested* counter that records whether a fused candidate was in fact constructed and ranked — so a run in which fusion never occurred cannot be reported as a merge win. We deliberately publish no headline win rate: the quantity is a property of the artifact’s key granularity and of proposal overlap, not of the mechanism (Section 8.4 reports it in situ for one workload).

5 Efficiency Analysis: Three Sources of Speedup

Wall-clock time in a self-evolution loop decomposes into rollout time, synchronization time, and held-out validation time. AgentDescent addresses each with a distinct mechanism; the third is specific to merge-based evolution.

5.1 Overlapping I/O-bound rollouts

Agent rollouts are dominated by time spent waiting on a model endpoint, so thread-level concurrency suffices to overlap them (Section 8.1 measures $5.8\times$ at eight threads on real API calls). The synchronous data-parallel runtime shards tasks across N workers over a common snapshot and merges at a round barrier. The barrier, however, imposes a ceiling independent of N : with i.i.d. rollout latencies t_i ,

$$T_{\text{round}}^{\text{sync}} = \mathbb{E}[\max_{i \leq N} t_i] + T_{\text{gate}}, \quad (4)$$

and reasoning-model latencies are heavy-tailed — a short answer and a long deliberation are the same call — which maximizes the gap between $\mathbb{E}[\max_i t_i]$ and $\mathbb{E}[t]$. The second term is a serial region in which all workers idle while the gate evaluates.

5.2 Barrier-free execution under bounded staleness

The asynchronous runtime (Algorithm 1) removes the round structure: workers produce continuously, a dedicated merger consumes continuously, and per-rollout cost approaches $\mathbb{E}[t]$ — the no-barrier discipline of asynchronous RL systems [4, 15] transferred to self-evolution. Its cost is staleness, which Eq. (2) makes first-class and per-diff, bounded by two mechanisms. A *lag budget* ρ (`async_ratio`) limits drift at the source: a worker refreshes its snapshot once v_{head} exceeds its snapshot version by more than ρ , and backpressure forces a global resynchronization if evidence accumulates while nothing commits. A *staleness policy* disposes of stale diffs at the merger: `FULL` admits them unconditionally (sound only where diffs compose); `GUARDED` rebases within $\eta \leq \alpha$ and discards beyond, in the spirit of bounded-staleness RL [4], paying for safety in discarded rollouts; `REFLECTIVE` always rebases and *re-verifies* — a stale diff is treated as unproven rather than invalid, and is discarded only if its claimed improvement fails to hold on the new head. Because ρ is

Algorithm 1 Barrier-free evolution (async_evolve); the synchronous runtime is the special case with a round barrier between the two loops.

```

1: Worker  $i$  (in parallel,  $i = 1..N$ ):
2: loop
3:   if  $v_{\text{head}} - v_i > \rho$  then refresh snapshot  $A_i \leftarrow$  ledger head;  $v_i \leftarrow v_{\text{head}}$ 
4:   end if
5:   sample task  $x$ ; rollout  $\tau \leftarrow \pi_{A_i}(x)$ ; reward  $r \leftarrow r(\tau, x)$ 
6:    $(d, e) \leftarrow S.\text{propose}(A_i, x, \tau, r)$ ;  $\text{buffer.put}(d, e)$  ▷ blocks on backpressure
7: end loop
8: Merger (single thread):
9: loop
10:   $\mathcal{B} \leftarrow \text{buffer.drain}()$ ; bucket  $\mathcal{B}$  by artifact
11:  for each bucket with  $|\mathcal{B}_a| \geq B$  or timed out do
12:     $\mathcal{B}_a \leftarrow \{d : \text{policy}(\eta(d), \alpha) \neq \text{DISCARD}\}$ , rebasing where directed ▷ Eq. (2)
13:    resolve conflicts (keep better of each contradicting pair);  $u \leftarrow \text{fuse}(\text{survivors})$ 
14:    if survivors contradict on a key and reflective merge enabled then  $u \leftarrow$  model-written union
15:    end if
16:    if audit passes and Eq. (3) holds on  $D_{\text{val}}$  then CAS-commit  $u$  to dev
17:    end if
18:  end for
19:  promote dev  $\rightarrow$  stable after  $K$  regression-free rounds
20: end loop

```

denominated in artifact versions, its appropriate value scales with rollout cost; the runtime emits a warning when a run discards the majority of its evidence.

5.3 Reducing validation cost

The third source of speedup is unavailable to the first two mechanisms: the gate itself. Every acceptance decision costs one sweep over D_{val} , and on live models this dominates the merger’s critical path — profiled on GEPA/HotpotQA, the merge thread’s busy time is gate evaluation to measurement precision. A serial or selection-based loop with P proposals pays $P \cdot |D_{\text{val}}|$ gate evaluations. Merging changes the count: a round’s n surviving diffs fuse into one candidate, so per-round gate cost falls from $n \cdot |D_{\text{val}}|$ to $|D_{\text{val}}|$; a diff discarded by the staleness filter never reaches the gate and costs zero evaluations; and in the asynchronous runtime, cards arriving during a merge enlarge the next batch rather than triggering additional merges. Measured at width 8 on a live model, merged validation reduces model calls by 28% relative to the serial arm at an identical rollout budget, with model-seconds per rollout falling $37.9 \rightarrow 23.8$ across arms (Section 8.1). Residual gate cost parallelizes independently of the worker pool (eval_concurrency), saturating at $|D_{\text{val}}|$.

The reduction presupposes that proposals *can* fuse, which fails exactly when the artifact’s key space is too coarse. An artifact held in a single key — e.g. a prompt as one instruction slot, GEPA’s setting [1] — makes every pair of concurrent proposals a write–write conflict; a keyed union then degenerates to retaining the better single proposal, i.e. best-of- n selection at best-of- n validation cost. *Reflective merge* restores the reduction: a model is prompted to write the *union* of the contradicting proposals — one candidate preserving each proposal’s intent — and the union passes through the same acceptance test as any candidate, so the model proposes and the gate decides. It is the spatial counterpart of the REFLECTIVE staleness policy (carry intent onto a changed base; let measurement decide), and the `contested` counter reports whether fusion genuinely occurred, so the saving is never claimed where only selection took place.

6 Governance by Blast Radius

Artifacts are assigned to layers by a declared `blast_radius`: **L2** (skills, prompts) merges under the full asynchronous pipeline; **L1** (harnesses, verifiers) serializes in-flight changes and routes every merge through the oracle; **L0** (the oracle, audit budget, merge permissions, safety constraints) is frozen and read-only to the loop. The frozen layer enforces the second disanalogy of Table 1: without it, a self-referential loop can converge to a verifier that accepts its own output. The same layering protects executed agent code, which runs behind a frozen test suite the candidate cannot rewrite; pristine tests are overlaid after workspace materialization, so weakening them at run time is likewise ineffective.

7 Case Study: Porting Eighteen Self-Evolution Algorithms

Because every component sits behind a seam (Section 3.3), porting a published algorithm requires a small set of plug-ins rather than a fork. Two routes cover all eighteen ports.

Route 1: a strategy, plus an optimizer where needed. A *benchmark-faithful* port supplies a strategy encoding the paper’s artifact as keyed diffs and, only when its search differs from the default pipeline, an `aggregator_factory`. The seven faithful ports illustrate the scale of the required code: ACE [28] is a play-book strategy whose curator *is* the default aggregator; GEPA [1] substitutes a per-instance Pareto optimizer; ADAS [5] and DGM [26] plug in keep-all archives with their own parent selection; EvoSkill a bounded top- K frontier; SkillOpt a strict edit gate; OpenEvolve [17] MAP-Elites islands [11]. Parent-selection and acceptance rules are extracted as named policy classes, so subsequent ports reuse them.

Route 2: a declarative policy bundle. The eleven microports and analogues (PromptBreeder, AFlow, Self-Refine, Reflexion, SICA, Gödel Agent, Voyager, SkillWeaver, Absolute Zero, R-Zero, Agent0) define no new classes: each is a `MethodPolicy` — selection rule, memory, acceptance shape, genome — over one shared runner, inheriting a common budget contract, dataset layer, and command line. A shared `--serial` flag reproduces the upstream algorithm’s own semantics (one worker, serial gate) as the control that any parallelization claim requires, and a shared rollout budget pins compared arms to equal work.

Fidelity is declared per port — *benchmark-faithful*, *mechanism microport*, *self-edit analogue*, *environment analogue*, or *inference analogue* — follows the released code where it diverges from the paper, and documents infrastructure boundaries explicitly; analogues are not to be cited as benchmark reproductions. All eighteen ports run under serial, synchronous, and barrier-free scheduling without modification, which is what permits the controlled scheduler comparison of Section 8.2.

8 Evaluation

Setup. All results are obtained from live models on real datasets: `deepseek-v4-flash` and `GLM-5.2` (reasoning models) via an OpenAI-compatible endpoint, on each algorithm’s own benchmark. Every number is produced by a named script in the repository with its exact reproduction command; absolute times depend on host and endpoint, so ratios are the reproducible quantity. We address four questions: **RQ1** — how much effective concurrency do the three mechanisms of Section 5 deliver against a faithful serial control? **RQ2** — does the speedup generalize across algorithms? **RQ3** — is learning behavior preserved under the most aggressive schedule? **RQ4** — does merging outperform best-of- N selection at equal budget?

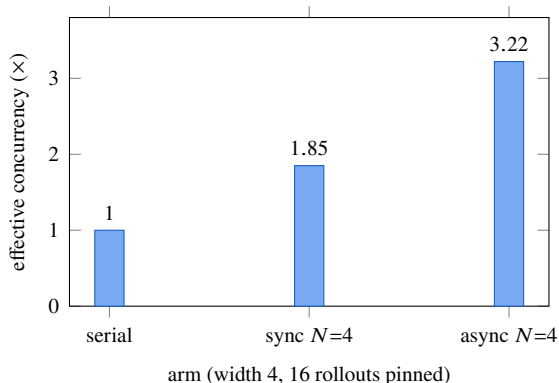


Figure 2: Effective concurrency (model-seconds $\div$ wall-clock) on the faithful GEPA port, HotpotQA, deepseek-v4-flash, 16 rollouts pinned on every arm. The serial control is the published loop exactly (one worker, serial gate: overlap 1.00 $\times$). The round barrier is the whole difference between the parallel arms: the same four workers reach 1.85 $\times$ with it and 3.22 $\times$ without it. Wall-clock: 609 s / 239 s / 140 s.

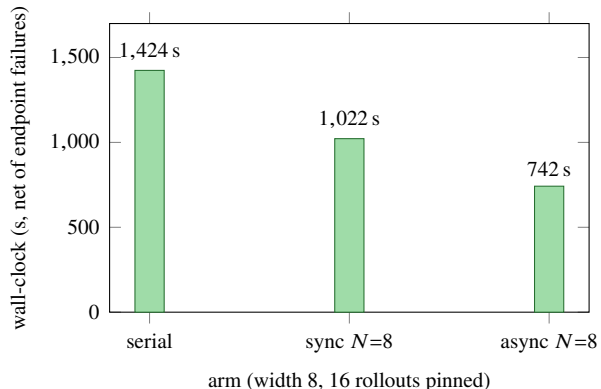


Figure 3: The same comparison at width 8 with GEPA’s own Pareto optimizer, GLM-5.2, wall-clock net of endpoint failures. Model calls: 97 (serial), 70 (sync — reflective merge collapses each round’s admissions into one sweep, -28%), 95 (async). Raw endpoint probes sustain 7–10 $\times$ concurrency, so the measured ladder ($1.31 \times / 1.80 \times / 2.89 \times$) is the engine’s structure, not the provider’s ceiling.

8.1 RQ1: Effective concurrency against a faithful serial control

We first verify the premise of Section 5.1: eight threads over one pool reduce a real GLM-5.2 API workload from 48.6 s to 8.3 s (5.8 $\times$; sub-linear under heavy-tailed latency), while the same threads on pure-Python CPU work yield 1.1 $\times$ — rollout overlap is real and the GIL is not a factor for I/O-bound rollouts.

The main experiment ports GEPA to its own benchmark (HotpotQA [1, 23]) with 16 rollouts pinned on every arm and a control that reproduces the upstream loop exactly: one worker *and* a serial gate, whose measured overlap is precisely 1.00 $\times$ (a control that evaluates concurrently is already partially parallel). At width 4 (Fig. 2), the synchronous arm reaches 1.85 $\times$ effective concurrency and the barrier-free arm reaches 3.22 $\times$ with the same four workers; wall-clock falls 609 $\rightarrow$ 239 $\rightarrow$ 140 s. At width 8 (Fig. 3), the two savings of Section 5.3 separate cleanly: the synchronous arm realizes merging as a *validation* saving — 70 model calls versus the serial arm’s 97 (-28%), as each round’s surviving diffs fuse into a single acceptance test — while the asynchronous arm realizes the same width as a *wall-clock* saving (1.92 $\times$). Model-seconds per rollout fall 37.9 $\rightarrow$ 27.7 $\rightarrow$ 23.8 across arms: the parallel arms not only finish sooner but evaluate less, which is the validation-cost reduction observed directly. Endpoint probes sustain 7–10 $\times$ concurrency, so no arm is provider-limited. Two caveats attach to the asynchronous arm: lacking a round boundary, it overran the pinned budget (19 vs. 16 rollouts), and at the default lag budget it discarded every stale proposal — zero-cost at the gate, but paid in proposals.

8.2 RQ2: Generality across algorithms

To exclude the possibility that the RQ1 result is specific to GEPA, the same scheduler comparison — serial, synchronous, barrier-free — is applied to all eleven declarative ports at an equal candidate budget, varying only the scheduler. Paired over $n = 33$ runs, the synchronous-parallel arm outperforms the serial arm by a median 1.36 $\times$ end-to-end and 1.89 $\times$ within the engine window, with all eleven methods showing a median improvement. The gap between the two figures is fixed cost outside the engine (dataset retrieval, final test scoring); the gap between 1.89 $\times$ and the worker count is the barrier and serial gate, i.e. precisely what the barrier-free runtime removes in Fig. 2.

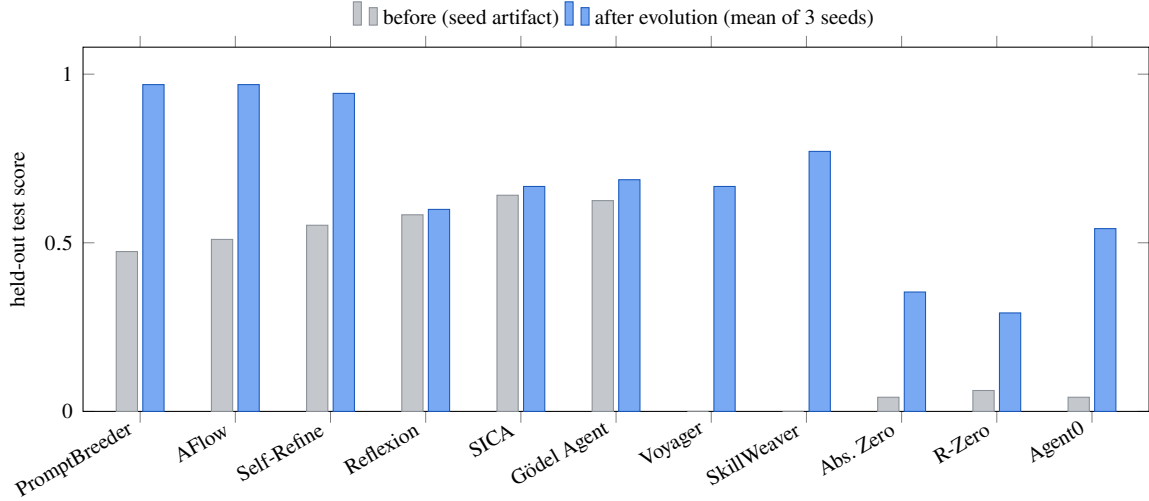


Figure 4: Held-out test score before and after evolution for the eleven microports and analogues, each run against deepseek-v4-flash under the *most aggressive* configuration — barrier-free async pipeline, 8 workers, Full staleness, 80 rollouts, 3 seeds (465–2,069 model calls per seed). All eleven improve. Between 3 and 10 of 240 proposals commit per method — the acceptance gate admits only strict held-out improvements, so sparse acceptance is the shape of the mechanism, not a low yield. Bars are comparable only within a domain: a gain is bounded by the headroom its baseline leaves.

8.3 RQ3: Quality preservation under the aggressive schedule

Throughput is meaningful only if learning is preserved, so quality is measured under the most aggressive configuration (barrier-free, 8 workers, FULL staleness). Figure 4 reports the eleven declarative ports at three seeds each: all improve their held-out test metric, with gains bounded by each benchmark’s headroom (the three GSM8K ports converge to 0.94–0.97 and differ in cost rather than quality). Among the benchmark-faithful ports, three representative results: ACE improves FiNER-139 validation 0.719 → 0.766 over 120 rollouts while curating a ten-bullet playbook; DGM’s best archived child repairs held-out bugs at 0.906 versus its seed agent’s 0.844 under a frozen pytest oracle; SkillOpt improves a hard SearchQA subset 0.053 → 0.211 with 3 of 43 proposed edits accepted. These single-seed runs establish end-to-end operation, not paper-scale comparisons. The highest-level interface (evolve_skill) on 40 HotpotQA rows raises held-out exact match from 0.167 to 0.583 over 338 model calls, committing one proposal.

8.4 RQ4: Merge-of- N versus best-of- N selection at equal budget

Throughput cannot distinguish merging from sampling-and-selecting, since N independent forks occupy N workers equally well; the discriminating quantity is held-out quality at an equal rollout budget. The largest such run to date — HotpotQA on GLM-5.2, three arms $\times$ three seeds, budget pinned in rollouts, reflective merge enabled so that contradicting proposals survive to fusion; 1,298 model calls and 11.3 hours of model time — yields a **null result**: merge-of-3 attains the highest median test score together with the widest spread, the intervals overlap, and the framework’s own separation test declines to declare a winner. The diagnosis is more informative than the verdict: at this budget the fused union was constructed twice in the entire experiment — most merges saw a single surviving diff — so the experiment does not measure a merging deficit; it measures a regime in which merging rarely fired. It also confirms the key-granularity condition of Section 5.3 from the opposite direction: without reflective merge, the merge arm on this single-key artifact would have reduced to best-of- N selection. Larger budgets on multi-key artifacts remain the open

measurement; the pinned-budget, seeded baseline harness (serial / fork with both oracle and dev-selected reporting / merge) ships with the framework.

9 Limitations

AgentDescent is a research reference implementation. (i) The throughput premise is a testable engineering hypothesis rather than established consensus, and the equal-budget quality question (RQ4) is currently unresolved. (ii) Turn-level checkpoint-and-resume of straggler rollouts and a stratified tail-canary held-out split are designed but not implemented. (iii) Most quality results carry three seeds and the benchmark-faithful ports one; they establish that the mechanisms operate, not effect sizes. Difficulty parameters calibrated on one model did not transfer under a model swap, whereas benchmark-intrinsic difficulty did. (iv) Several earlier project-internal measurements proved irreproducible or favorably defined and were re-measured and corrected in place; every number reported here is generated by a named script. We regard this correction trail as part of the contribution: the evaluation of a self-improving system should satisfy the standard its own acceptance gate imposes on diffs.

10 Conclusion

The distributed-training stack admits a coherent counterpart in agent self-evolution once its necessary disanalogy — diffs do not add — is taken seriously and merging is constructed as a discrete-space optimizer. Efficiency decomposes into three sources, of which the distinctive one is reduced validation cost: merging collapses N acceptance tests into one, reflective merge extends the reduction to conflicting proposals, and on live models the resulting ladder reads $1.85\times$ with the barrier, $3.22\times$ without, and 28% fewer model calls against a faithful serial control. Eighteen published algorithms adapt as plug-ins rather than forks. Whether merging also improves quality over best-of- N selection at equal budget remains open; the instruments to resolve it ship with the framework.

Acknowledgments

The algorithm ports were contributed by chendanyang (the seven benchmark-faithful ports) and cyanneko (OpenEvolve and the eleven microports and analogues).

Reproducibility

All code is available at <https://github.com/Birfy/agentdescent> (MIT license; pip install agentdescent; the core engine has no required dependencies). Generating scripts: examples/efficiency.py (thread overlap), bench/matrix_run.py and examples/gepa/ (RQ1), bench/candidate_methods.py (RQ2, RQ3), bench/baselines_run.py (RQ4). Every port supports `--dry-run` (no API key or model call) and carries an offline test suite; datasets are retrieved from canonical sources by a dependency-free data layer.

References

- [1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.

- [2] CoEvoSkills authors. Self-evolving agent skills via co-evolutionary verification. *arXiv preprint arXiv:2604.01687*, 2026.
- [3] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.
- [4] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.
- [5] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- [6] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Fang, Jiaxin Huang, Tony Sanders, and Dong Yu. R-Zero: Self-evolving reasoning LLM from zero data. *arXiv preprint arXiv:2508.05004*, 2025.
- [7] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, Hyoungho Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. GPipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in Neural Information Processing Systems*, 32, 2019.
- [8] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [9] Mu Li, David G Andersen, Jun Woo Park, Alexander J Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J Shekita, and Bor-Yiing Su. Scaling distributed machine learning with the parameter server. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 583–598, 2014.
- [10] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2023.
- [11] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- [12] Feng Niu, Benjamin Recht, Christopher Ré, and Stephen J Wright. HOGWILD!: A lock-free approach to parallelizing stochastic gradient descent. *Advances in Neural Information Processing Systems*, 24, 2011.
- [13] Alexander Novikov, Ngân Vŭ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. AlphaEvol: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [14] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.
- [15] ROLL Team. ROLL Flash: Accelerating RLVR and agentic training with asynchrony. *arXiv preprint*, 2025.
- [16] Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. *arXiv preprint cs/0309048*, 2003.

- [17] Asankhaya Sharma. OpenEvolve: an open-source evolutionary coding agent. <https://github.com/codelion/openevolve>, 2025.
- [18] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.
- [19] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [20] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [21] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, pages 23965–23998, 2022.
- [22] Peng Xue, Cheng Wu, Zhen Lin, et al. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. *arXiv preprint*, 2025.
- [23] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.
- [24] Xunjian Yin, Xinyi Wang, Liangming Pan, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursive self-improvement. *arXiv preprint arXiv:2410.04444*, 2024.
- [25] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. *Advances in Neural Information Processing Systems*, 33:5824–5836, 2020.
- [26] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.
- [27] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. AFlow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024.
- [28] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025.
- [29] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Yang Yue, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, et al. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.
- [30] Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, et al. SkillWeaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.